

COFORGE GRADUATE ENGINEER TRAINEE PROJECT

LuxeFlats

Flat Rental Microservices Platform

Presented By: Tanya Gupta, Harsh Sahu, Kshitij Goel

Role: Graduate Engineer Trainee

Technology Stack: Java | Spring Boot | React + Vite | MySQL

Emp Code: 85011019

Email: Tanya.Gupta@coforge.com

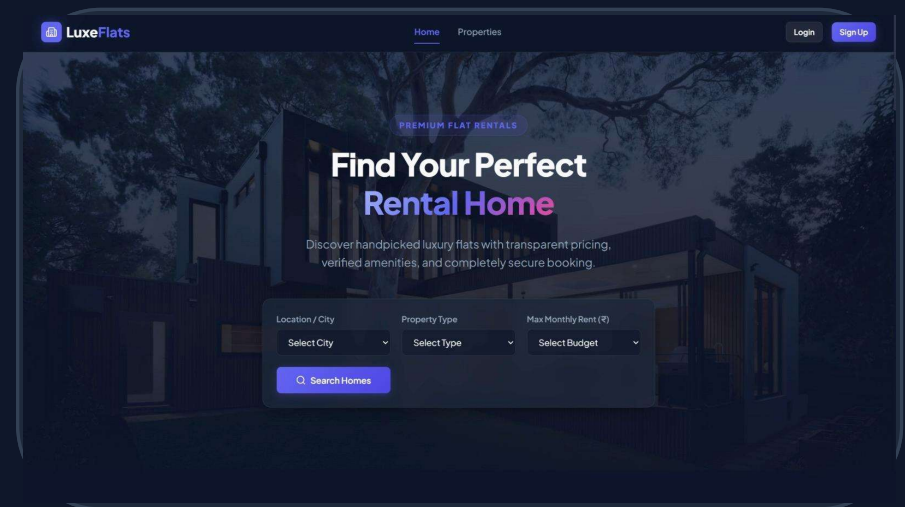


Table of Contents

01. Project Overview

- Introduction to the Flat Rental System
- Problem statement and proposed solution
- Key objectives and project scope

02. Technology & Architecture

- Technology Stack: Java, Spring Boot, React + Vite, MySQL
- Microservices: User, Flat & Rental Services
- Architecture: Eureka Service Discovery + API Gateway

03. System Implementation

- Development of REST APIs using Spring Boot
- Integration of React + Vite frontend with backend
- Database operations using MySQL and JPA/Hibernate

04. Results & Features

- User, flat, and rental management
- Flat availability and rental operations
- Integrated frontend with microservices backend

05. Future Scope & Conclusion

- Add secure authentication and online payment
- Enable Docker/Kubernetes and cloud scaling
- Scalable architecture for future enhancements

06. Testing & Performance

- API testing using Postman
- Validation of service communication and database operations

Problem Statement & Project Objectives

The Problem

- Traditional rental processes involve manual communication and scattered information.
- Difficult to efficiently track flat availability and rental records.
- Lack of a centralized platform for managing rental operations.

Key Objectives

- Simplify flat discovery and rental management.
- Centralize user, flat, and rental information.
- Build a scalable and maintainable microservices-based system.



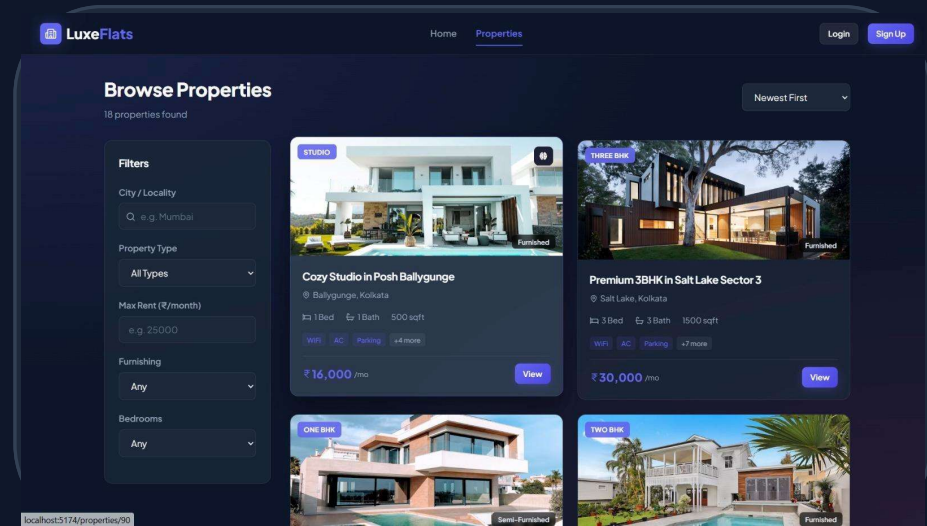
Project Overview & Key Idea

System Definition

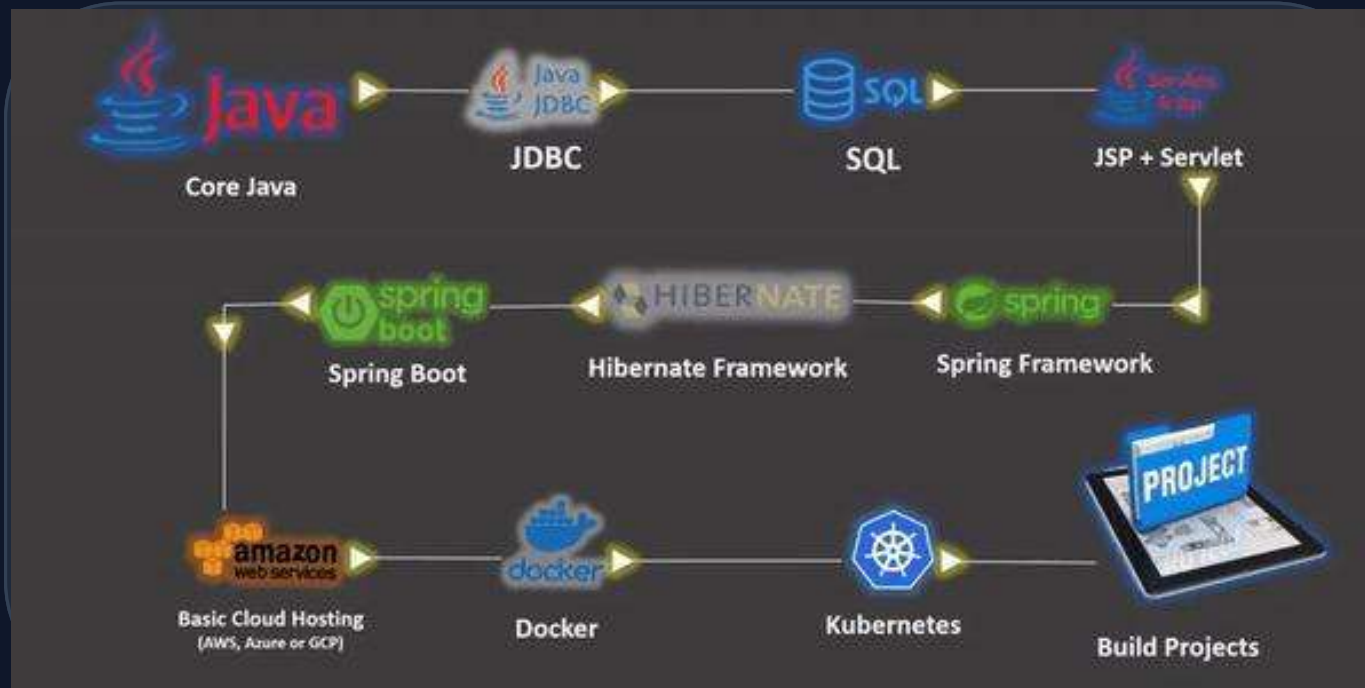
- The Flat Rental System is a web-based application designed to simplify and manage the complete flat rental process.
- A React + Vite frontend provides the user interface, while Spring Boot REST APIs handle backend operations.
- It provides a centralized platform for managing users, flats, availability and rental information.
- The system follows a microservices architecture, where different business functionalities are developed as independent services.

Core Architecture Idea

- Users interact with the system through the React + Vite application.
- Requests are routed through the API Gateway to the appropriate microservice.
- Eureka Server enables service discovery between the microservices.
- Data is stored and managed using MySQL databases.



Development Tools & Environment Setup



Development Stack: Core Java -> JDBC -> SQL -> JSP/Servlet -> Spring Framework -> Hibernate -> Spring Boot.
Deployment Pipeline: Basic Cloud Hosting (AWS/Azure/GCP) -> Docker Containers -> Kubernetes Orchestration.

Project Folder Directory Structure



Service Module Breakdown

- api-gateway: Routes incoming client calls to targeted downstream ports.
- flat-service (property-service): Manages flat properties data, images, and locality parameters.
- rental-service (booking-service): Coordinates lease agreements, check-in validation, and state changes.
- service-registry (eureka): Dynamic service discoverer mapping container IPs.
- user-service (auth-service): Handles credential registration, password hashing, and user role records.

Project Technology Stack

FRONTEND

- React
- Vite
- HTML5 & CSS3
- TypeScript
- Axios Client



MICROSERVICES & CLOUD

- Spring Cloud Gateway
- Eureka Server
- OpenFeign Client
- Spring Cloud Infrastructure

BACKEND

- Java
- Spring Boot
- REST APIs
- IOC & DI Container

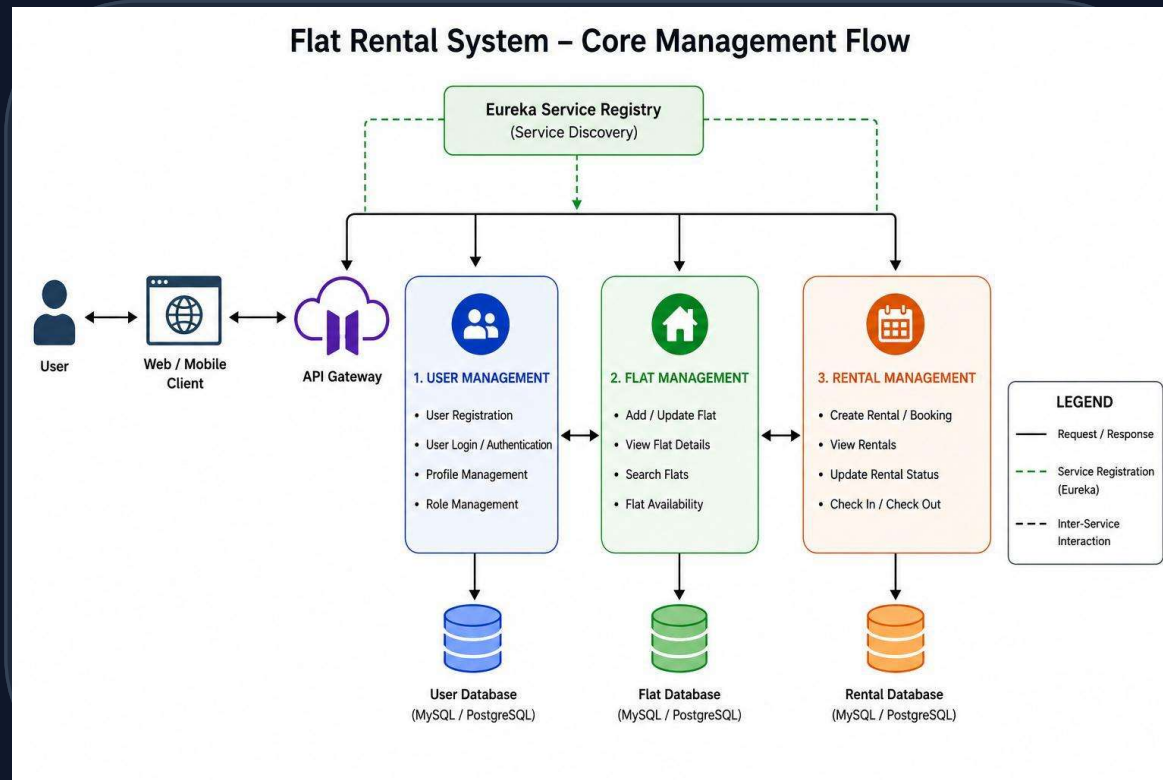
PERSISTENCE

- MySQL Database
- Spring Data JPA
- Hibernate Framework
- Repository Mappings

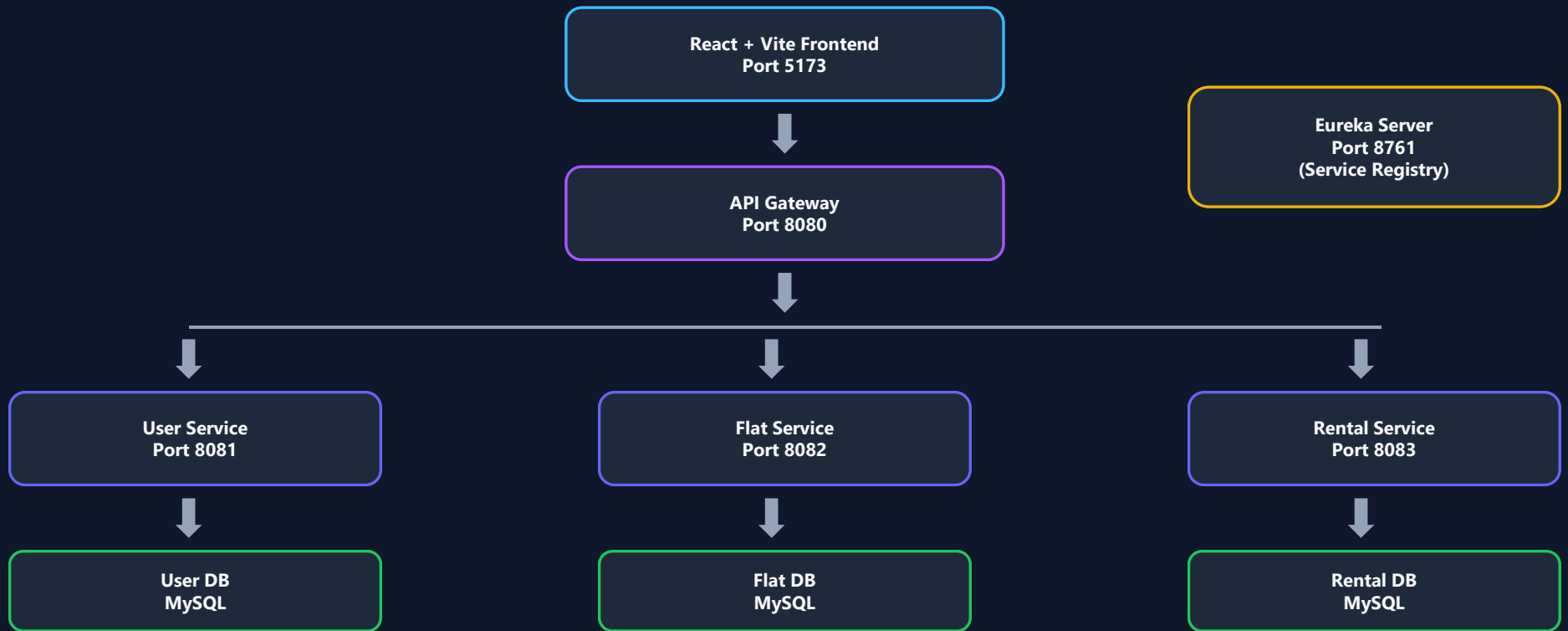
DEV & OPS

- Maven Build Tool
- Postman API Test
- Git / GitHub
- Docker & Kubernetes

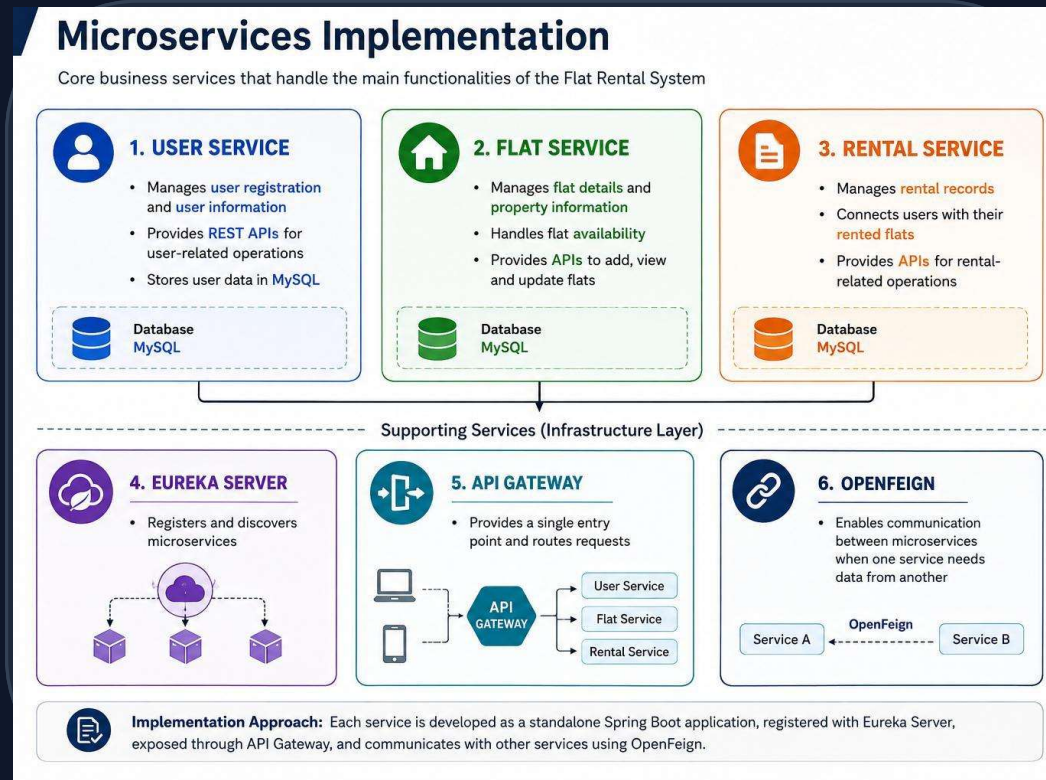
Flat Rental System - Core Management Flow



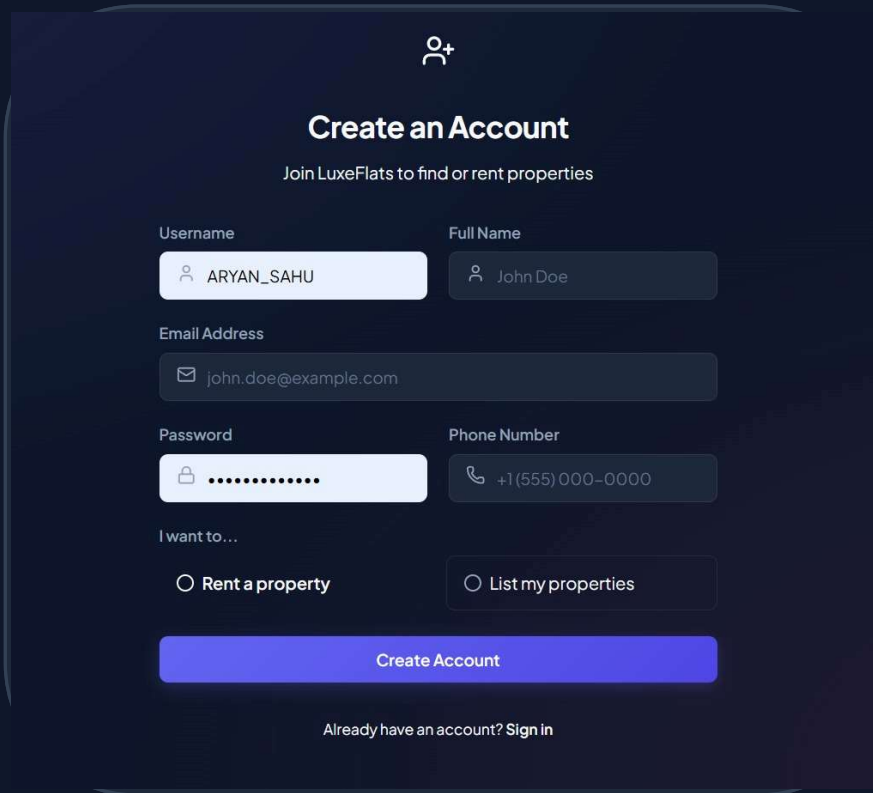
Microservices System Architecture



Microservices Functional Mapping



Project Snapshots - User Registration



A screenshot of a user registration form titled "Create an Account" with a subtext "Join LuxeFlats to find or rent properties". The form includes input fields for Username (filled with "ARYAN_SAHU"), Full Name (filled with "John Doe"), Email Address (filled with "john.doe@example.com"), Password (masked with dots), and Phone Number (filled with "+1(555) 000-0000"). Below these fields are two radio buttons under the heading "I want to...": "Rent a property" (selected) and "List my properties". A large blue "Create Account" button is at the bottom, followed by a link "Already have an account? Sign in".

Create an Account
Join LuxeFlats to find or rent properties

Username: ARYAN_SAHU

Full Name: John Doe

Email Address: john.doe@example.com

Password:

Phone Number: +1(555) 000-0000

I want to...

☒ Rent a property

☐ List my properties

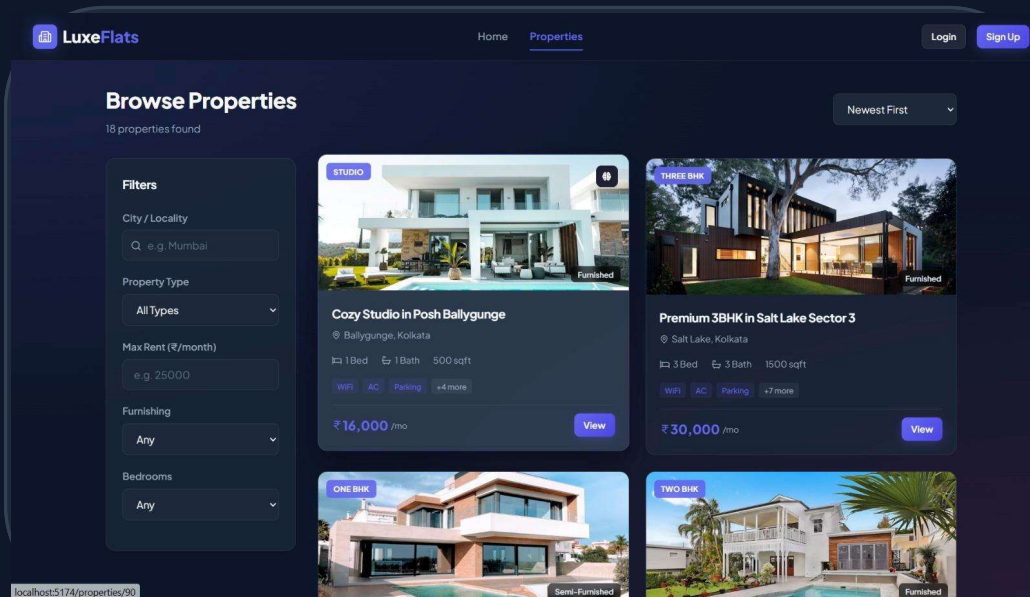
Create Account

Already have an account? [Sign in](#)

Registration Workflow

- Provides a secure and intuitive user registration screen.
- Differentiates between roles: Renting a property (Tenant) and Listing a property (Owner).
- Validates fields client-side before forwarding HTTP requests to auth-service.
- Standard credentials initialized for immediate end-to-end testing: tenant/password123, owner/password123.

Project Snapshots - Property Discovery



Properties Marketplace

- Renders property listings inside glassmorphic layout cards.
- Allows dynamic client-side filtering by: City/Locality, Property Type (Studio, 1 BHK, 2 BHK, etc.), Rent Budget range, and Furnishing status.
- Displays responsive cards with price metrics, active amenities, and a link to view details.
- Features realistic property images loaded dynamically from Unsplash CDNs.

Project Snapshots - Booking Requests

Request Booking



Cozy Studio in Posh Ballygunge
Ballygunge, Kolkata
1 Bed 1 Bath Furnished

Monthly Rent	₹ 16,000
Security Deposit	₹ 35,000

Select Dates

Move-in Date
mm/dd/yyyy

Move-out Date
mm/dd/yyyy

Confirm Booking Request

No payment required now. Booking is subject to owner approval.

ⓘ Your booking will be in **PENDING** status until the owner confirms it. Payment will be collected after confirmation.

Booking stays Flow

- Allows tenants to pick custom move-in/move-out dates on an interactive calendar.
- Automatically displays property pricing parameters, including monthly rent and security deposits.
- Triggers validation on dates (move-out must be after check-in, check-in must be present/future).
- Saves the initial contract with status **PENDING**, waiting for owner approval.

Testing & Performance Overview

API Testing

- Tested REST APIs using Postman
- Verified GET, POST, PUT, and DELETE operations
- Checked HTTP response codes and response DTO schemas

Integration Testing

- Verified communication between React + Vite and Spring Boot
- Validated API Gateway dynamic request routing
- Confirmed microservice registry with Eureka Server

Database Testing

- Verified CRUD operations directly inside MySQL
- Validated Hibernate-based insertion, retrieval, and updates
- Ensured API payloads matched MySQL table records

Performance & Reliability

- Independent services minimize module-to-module dependencies
- Centralized gateway routing simplifies CORS and filter configs
- Microservices architecture allows services to scale independently

API Validation & Testing Approach

API Testing Overview

- API Testing is a software testing approach that focuses on validating the functionality, reliability, performance, and security of Application Programming Interfaces (APIs).
- Unlike GUI testing, API testing verifies the communication between different software components by sending requests and validating responses without interacting with the user interface.
- It ensures that APIs function correctly, exchange accurate data, and meet the specified business and technical requirements. Popular tools such as Postman, Rest Assured, and JMeter are widely used for API testing.

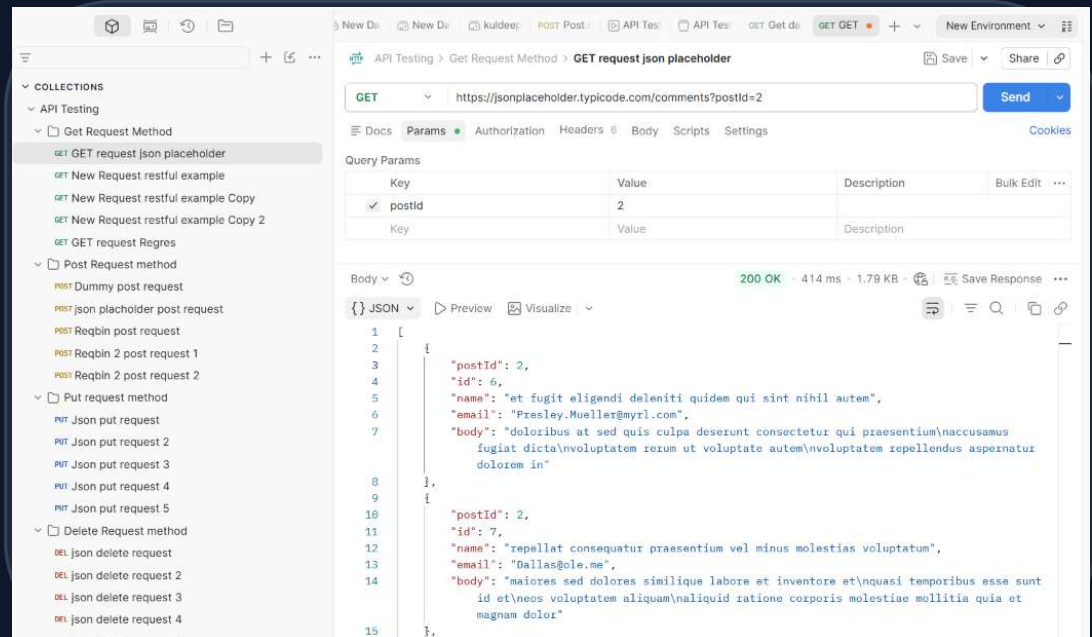
Importance of API Testing

- Validates the functionality and reliability of APIs.
- Ensures accurate data exchange between applications.
- Detects defects early before the user interface is developed.
- Improves application performance and response validation.
- Supports functional, regression, and integration testing.
- Enhances software quality by verifying API contracts and business logic.
- Facilitates Continuous Integration and Continuous Delivery (CI/CD) through automated API testing.

API Validation - GET Request

Postman GET Request Analysis

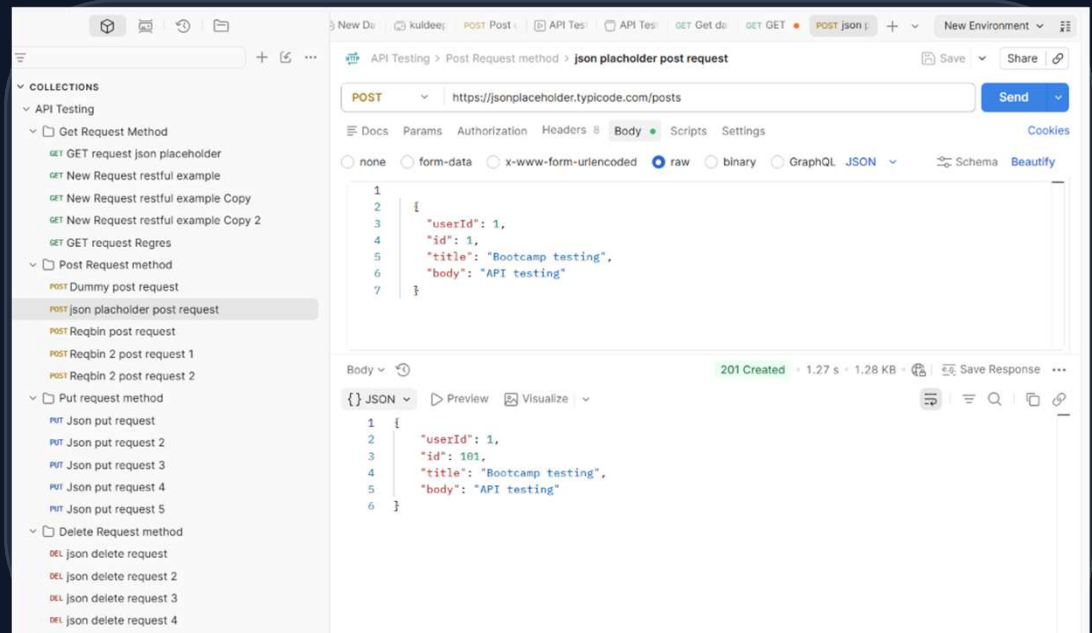
Used to retrieve resources or data lists from a specified URL endpoint without modifying any database records.



API Validation - POST Request

Postman POST Request Analysis

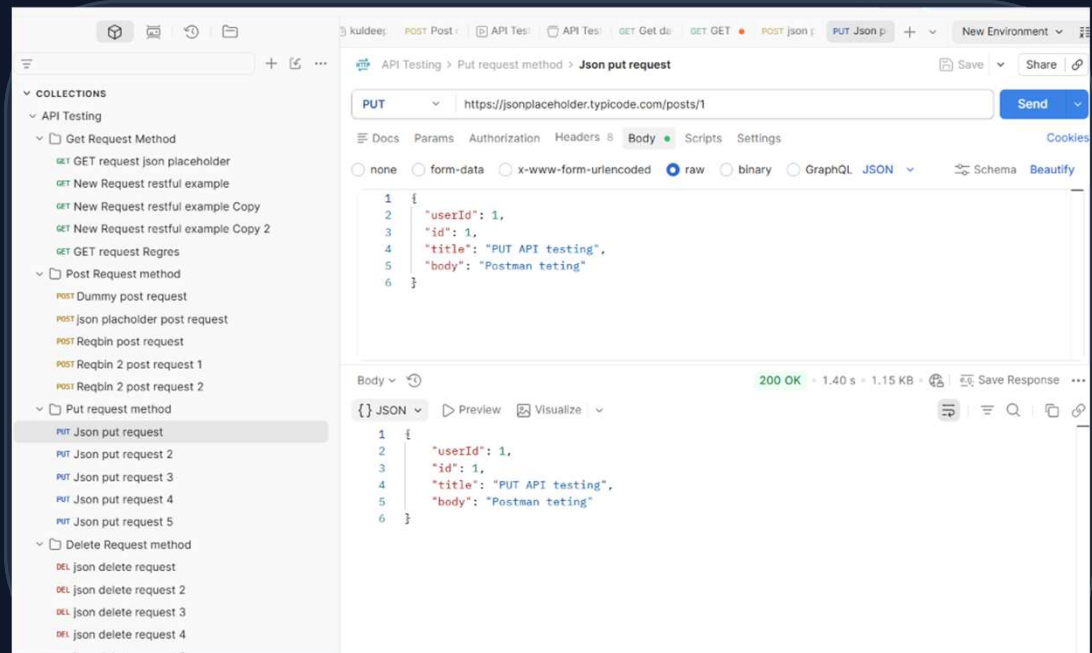
Used to submit data payloads to create new resources on the server, resulting in a database insertion (returns 201 Created).



API Validation - PUT Request

Postman PUT Request Analysis

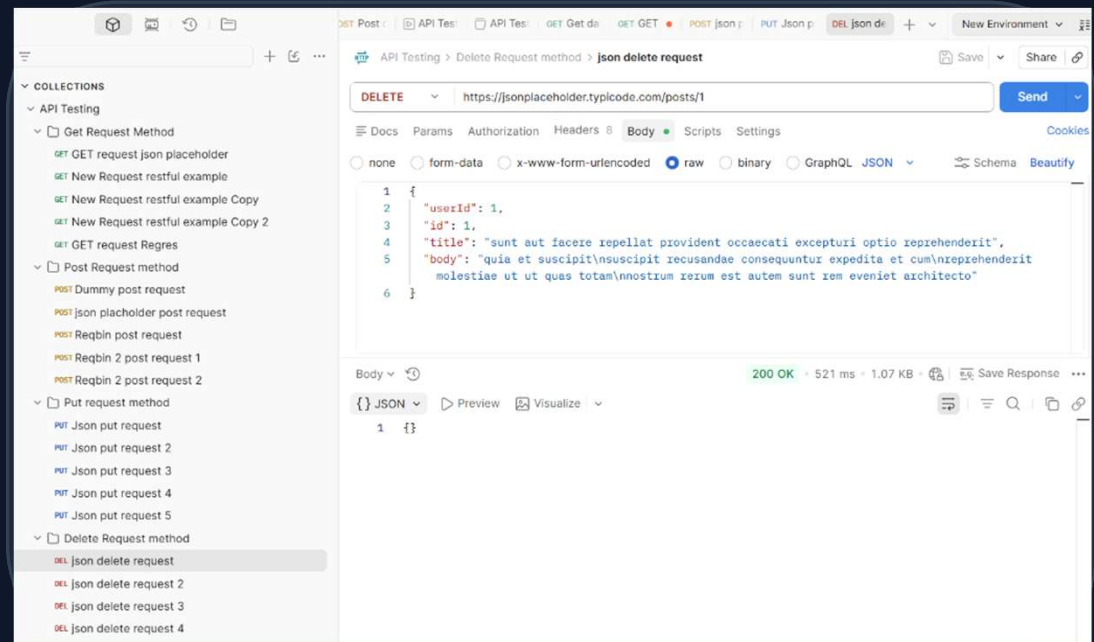
Used to update an existing resource with a complete new representation payload, replacing the database row fields.



API Validation - DELETE Request

Postman DELETE Request Analysis

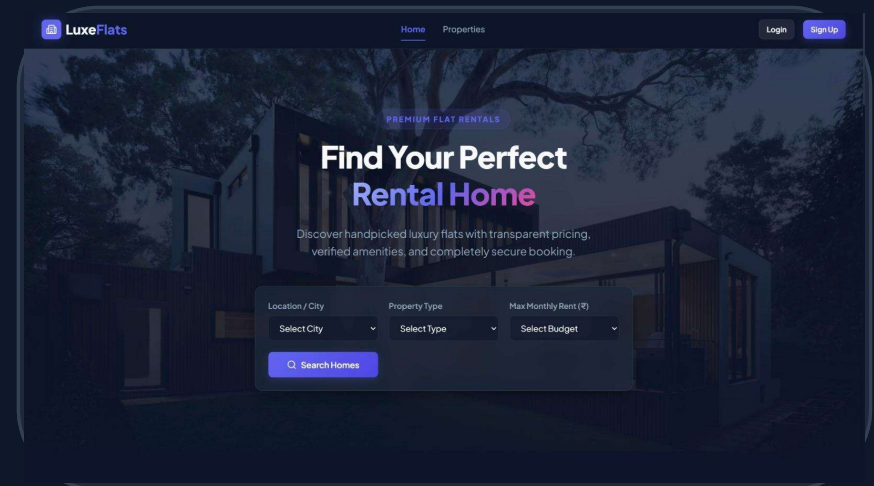
Used to remove a specific resource from the database based on target ID parameters (returns 204 No Content).



Project Conclusion & Takeaways

Key Accomplishments

- Developed a highly modular, microservices-based Flat Rental System for managing users, flats, and bookings.
- Successfully implemented User, Flat, and Rental Services using independent Spring Boot applications.
- Integrated the frontend using React + Vite, API Gateway routing, Eureka service registry, and MySQL databases.
- Verified all endpoints, security filters, and inter-service communications using Postman.
- Provides a highly scalable, maintainable, and robust architecture suited for cloud deployment.



Thank You

"Quality is never an accident; it is always the result of intelligent effort."

Presented By: Tanya Gupta, Harsh Sahu, Kshitij Goel
Graduate Engineer Trainees